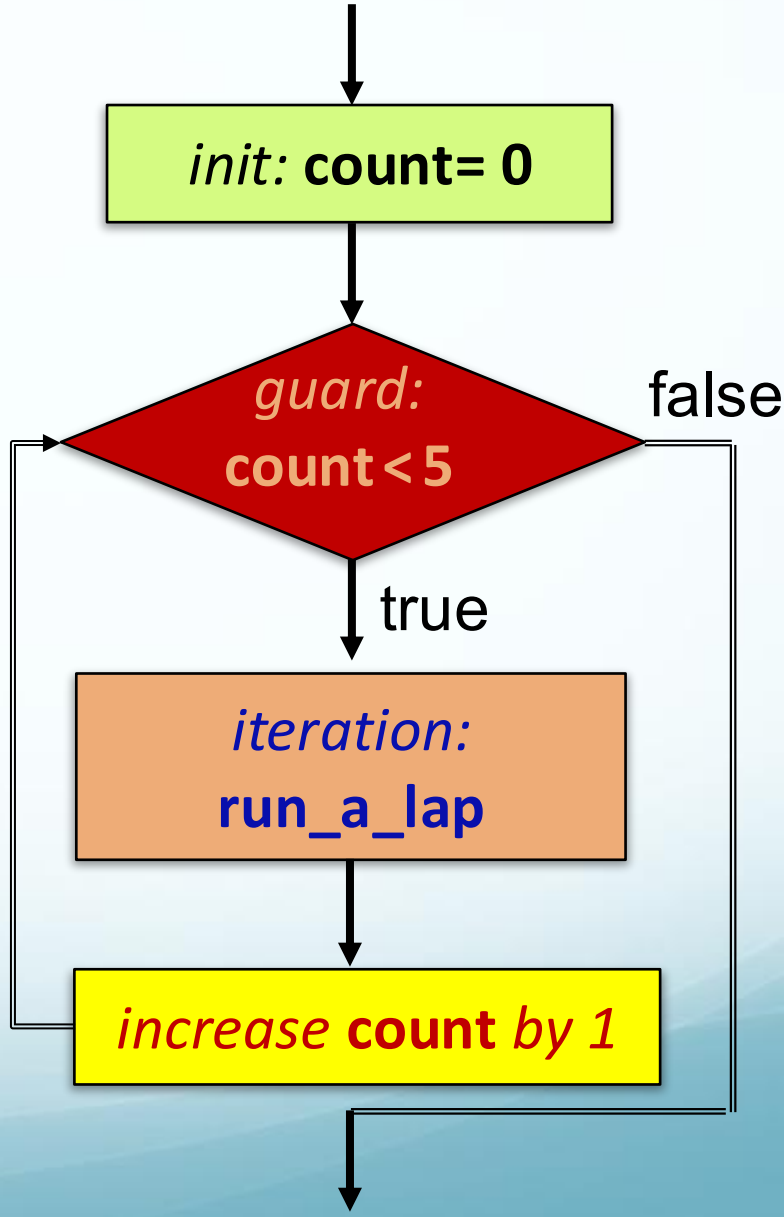
COMP20005 Workshop Week 4

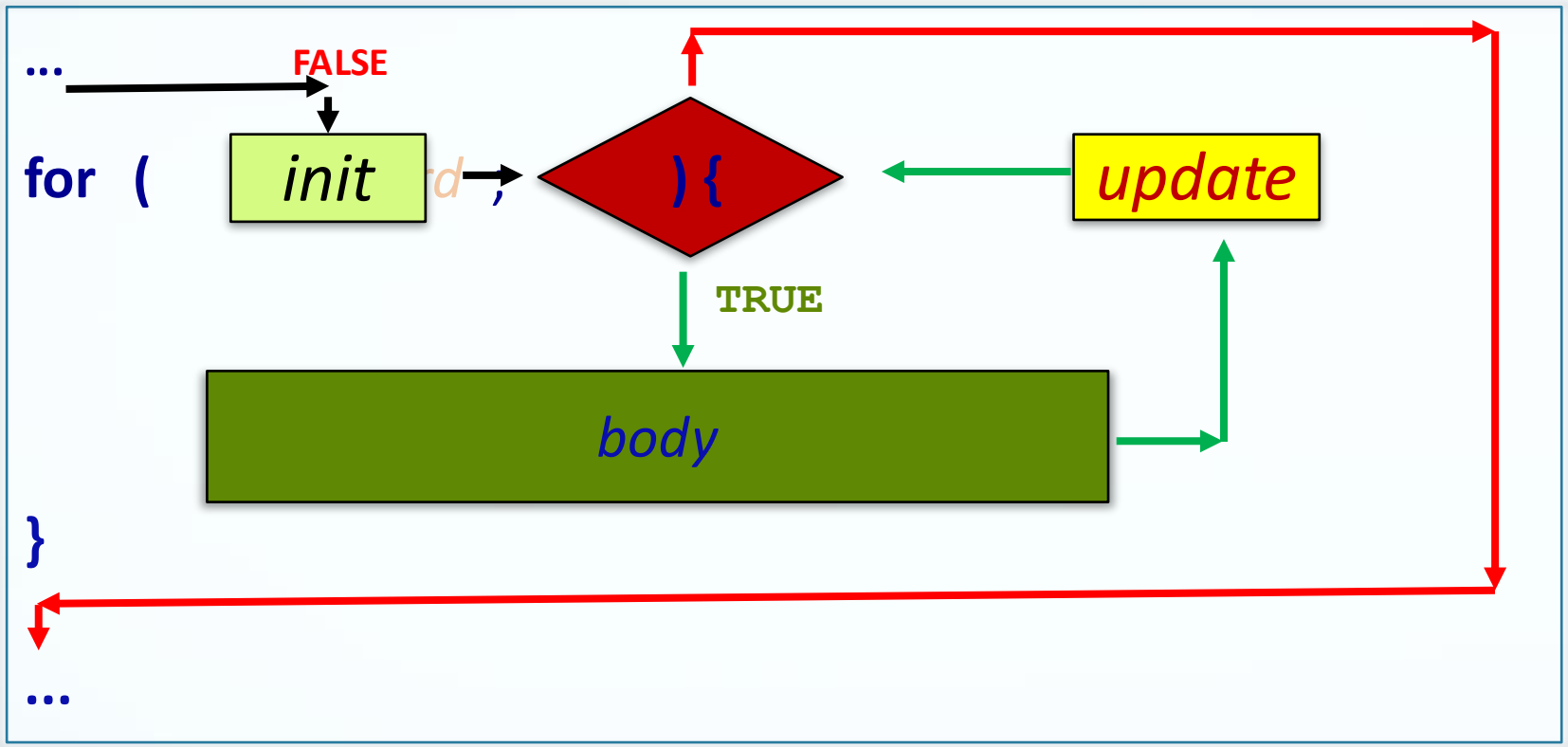
loop-de-looping

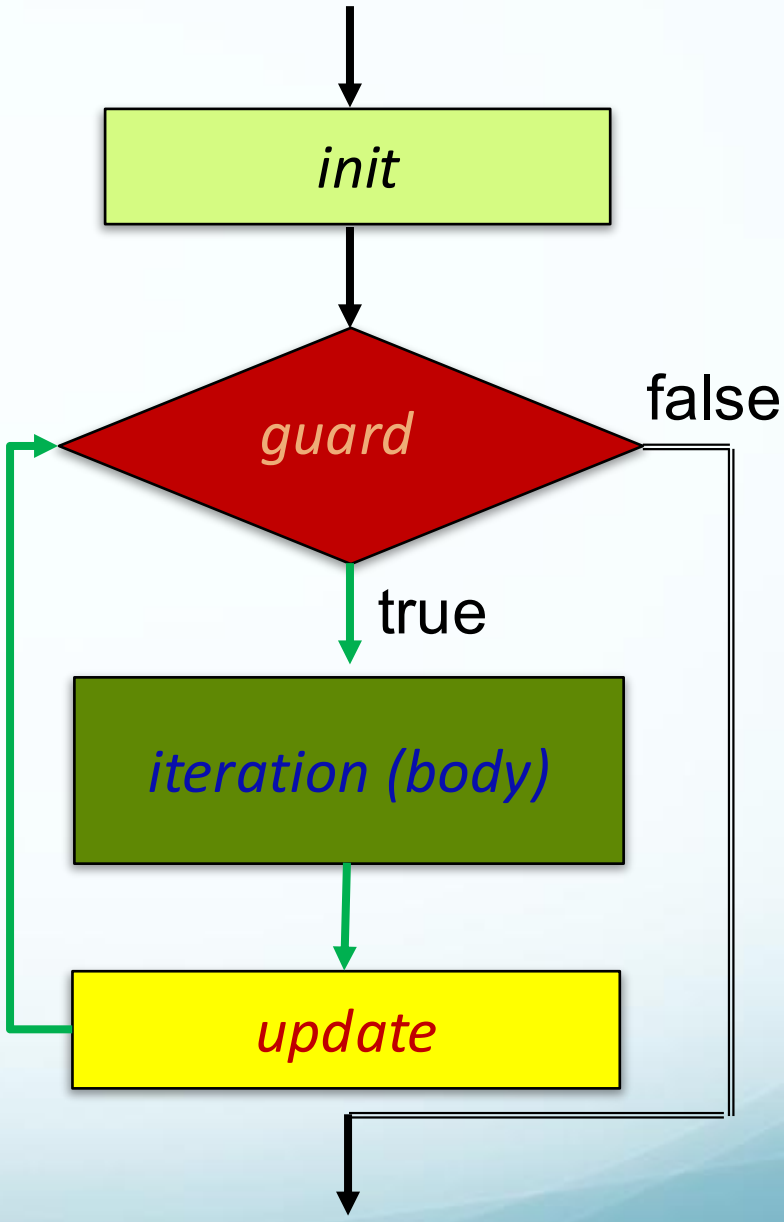
1 **LAB**	Q&A: loops; Discuss **`Ex 4.01`** and **`4.02`** Do Together: Discuss & Start **`Ex 4.05`** Discuss then Implement: **`Ex 4.09`** min requirement: finish **`4.05`**, **`4.09`** Better if also complete: **`4.04 then 4.06 then (4.07 or 4.08)`** Complete other exercises from Chapter 4
Future	**MST** (16 APR = Thursday Week 6): covers chapters 1—5 (ie. till the end of functions)

Running exactly 5 laps: the for loop

```
for ( count= 0 ; ) {
    run_a_lap();
    count++
}
```







Each part of the for loop (e.g., each box) can be omitted, but the two semicolons are required:

for (; ;)

Note: If the guard (condition) is left empty, it is treated as TRUE.

The for ... loop

Examples: use the for loop to compute:

1. S= 1/1 + 1/2 + 1/3 + . . . + 1/10
2. sum_odd= the sum of all positive odd integers up to n

Review: few different versions for The for … loop

$$S = 1^2 + 2^2 + 3^2 + 4^2 + 5^2$$

```
for (S=0, i= 1; i <= 5; i++) {
   S += i*i;
}
```

```
S=0, i=1;
for (  ; i <= 5; i++) {
   S += i*i;
}
```

```
S=0, i=1;
for (  ; i <= 5;   ) {
   S += i*i;
   i++;
}
```

```
for ( S=0, i=0 ; i < 5; i++) {
   S += (i+1)*(i+1);
}
```

```
for (S=0, i=5  ; i > 0 ; i-- ) {
   S += i*i;
}
```

continue (== go to <update>) & break (==exit loop)

```
for ( s=0, i=1  ; i<=100  ;  i++ ) {

    f (i==2) {
        continue;
    }

    s += i*i;
    f (i==4) {
        break;
    }

}

printf("the end: i=%d, s= %d\n",i,s);
```

s= 0, i= 1

1 <=100: s= update i=

[Class Discussion] Ex 4.01 a-c)

Trace the action of the loop, and determine the values printed out by the printf statement. Assume that all variables have been declared to be of type int.

a)

```
for (i=0; i<20; i=i+3) {
  printf("%2d\n", i);
}
```

b)

```
for (i=1; i<2000000; i=2*i) {
      printf("%7d\n", i);
  }
```

c)

```
sum = 0;
for (i = 1; i < 10; i++) {
  sum = sum + i;
  printf("S(%2d) = %2d\n", i, sum);
}
```

Ex 4.01 d) (modified)

```
for (i= 0; i < 3; i++) {

  for  (j= i+1; j < 8; j += 3) {
    printf ("i= %d, j= %d\n", i, j);
  }

}
```

i= , j=

Ex 4.01 e) (modified)

```
for (i= 1; i < 5; i++) {
  for  (j= i+1; j < 8; j += 3) {
    if (i+j == 7) {
      break;
    }
    printf ("i= %d, j= %d\n", i, j);
  }
}
```

i= , j=

Ex 4.01 f)

```
j = 5;
for (i= 0; i < j; i++) ; {
    printf ("i= %d, j= %d\n", i, j);
}
```

i= , j=

How many lines printed out?

Ex 4.01 g)

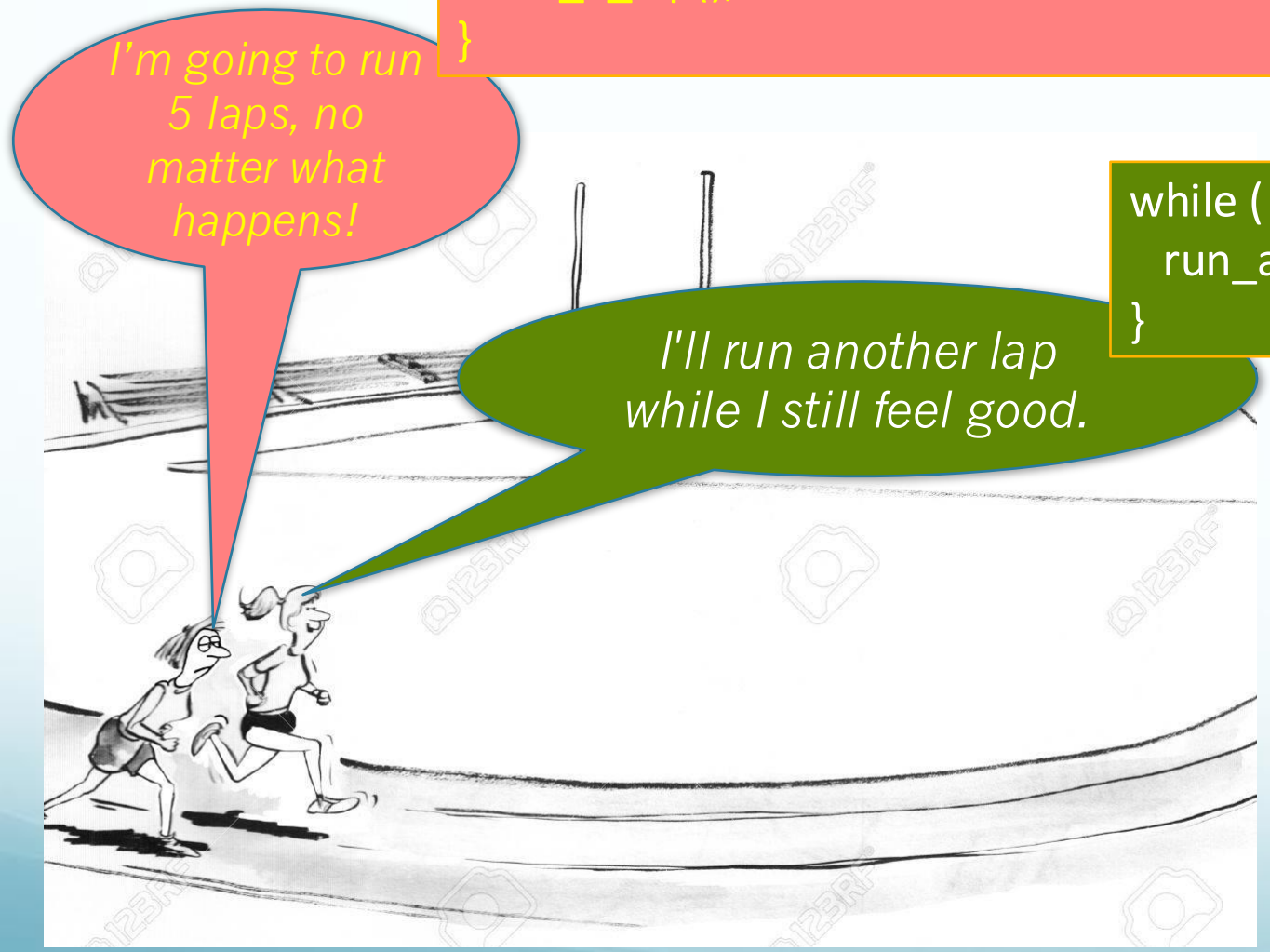
```
j = 5;
for (i= 0; i < j; j++) {
    printf ("i= %d, j= %d\n", i, j);
}
```

i= , j=

How many lines printed out?

Loops: the while loop compared with the for loop

```
for (count= 0; count < 5; count++) {
   run_a_lap();
}
```




```
while ( feel_good() ) {
   run_a_lap();
}
```

A for loop is preferable when the number of iterations is known or tracked by a counter.

image: adapted from https://www.123rf.com/photo_57742073_sports-cartoon-about-the-last-lap-in-the-race.html

Running exactly 5 laps: the while loop vs. the for loop

```
count= 0
while ( count < 5 ) {
        run_a_lap();
        count++ ;
}
```

```
count= 0
for ( ; count < 5 ;) {
        run_a_lap();
        count++ ;
}
```

```
for ( count= 0 ; ) {
        run_a_lap();
}
```

count++

iteration

all three versions are equivalent!

the while ... loop

- could be considered as a special case of the for loop, when *init* and *update* are both empty

```
while ( condition ) {
    statements
}
```

same as

```
for ( ; condition ; ) {
    statements
}
```

```
for (init; condition ; update) {
    statements
}
```

same as

```
init
while ( condition ) {
  statements
  update
}
```

Loops: while ... do ... vs. do ... while ...

```
while ( guard ) {
    statements;
}
```

not always the same as

```
do {
    statements;
} while (guard);
```

[Quick Discussion] Ex 4.02

Give a general construction that shows how any `do` *statement can be converted into an equivalent* `while` *statement.*

do { statement; **} while (**guard**);**	**while (** guard **) {** statement; **}**

Start-Together in Ed: Ex 4.05

Design and implement a program `grapher.c` *that reads integers and draw a simple graph. Assume that all of the values read are between 1 and 70. Example:*

```
./grapher
Enter integers between 1 and 70 inclusive:
3  7  11
 3 |***
 7 |*******
11 |***********
```

How to approach Ex 4.09

Ex4.09: Compute the next prime number

Task

Write a program `nextprime` that calculates the *next* prime number after a given value.

Sample output

In the output below, 11 is the *next* prime after 8.

```
Enter an integer value: 8
The next prime is     : 11
```

Time for fun: Quick Test

Note: *In all questions, all variables are pre-declared as* int.

Q1: What are the values of s, i, and c after the following statement:

```
for (s=0, i=0, c= 0; i<5; i++) {
    s += i;
  c++;
}
```

A	s= 10, i= 5, c= 5
B	s= 10, i= 5, c= 4
C	s= 15, i= 6, c= 6
D	none of the above

Time for fun: Quick Test

Q2: How many lines and numbers are printed by the following segment:

```
for (i=0; i<2; i++) {
  for (j=0; j<3; j++) {
      printf("%d ", i*j);
  }
  printf("\n");
}
```

A	6 lines, 6 numbers
B	3 lines, 12 numbers
C	2 lines, 6 numbers
D	none of the above

Time for fun: Quick Test

Q3: How many lines and numbers are printed by the following segment:

```
for (i=0; i<5; i++) {
  for (j=0; j<4; j++) {
    if ( j >i ) continue;
    printf("%d ", i*j);
  }
  printf("\n");
  if (i==2) break;
}
```

A	5 lines, 20 numbers
B	3 lines, 6 numbers
C	2 lines, 3 numbers
D	none of the above

Time for fun: Quick Test

Q4: Which fragment compute $s = 1^2 + 2^2 + \ldots + n^2$?

A	i=0; while (i<=n) s += i*i;
B	for (s=0; n > 0; n--) s += n*n;
C	for (i=1; i<=n; i++) s += i*i;
D	for (s=0,i=1; i<n; i++) s = s + i*i;

Quiz: Check your answers

Q1. B. s=10, i=5, c=4.

Q2. C. 2 lines, 6 numbers. Each iteration of the i loop prints one line; within each line, the j loop prints 3 numbers.

Q3. B. 3 lines, 6 numbers. The i loop runs for 3 iterations (i = 0, 1, 2). The j loop runs i+1 times, printing a total of 1 + 2 + 3 = 6 numbers.

Q4. B. for (s=0; n > 0; n--) s += n*n;

- A and C: Incorrect because s = 0 is not initialised.
- D: Incorrect because it misses the final term (n^2) in the sum.
- B: Correct. Although it computes the sum in reverse order, it is mathematically equivalent. Note that the original value of n is lost after the loop executes.

Lab

min requirement: finish **`4.05`**, **`4.09`**

others: **`4.04`**

then: **`4.06 (4.08 or 4.07)`** *and others exercises in* **`Chapter 4 ...`**

Remember:

- Discuss with your classmates
- Show Anh one of your programs if you haven't done so

At home: Preparing for the Mid-Semester Test (Thursday Week 6) by

- Reviewing lectures
- Practicing programming *on paper* using exercises in Chapter 1 – Chapter 5,
- Bringing questions to the next week workshop
- Making sure about the MST: Where, When, What Rules